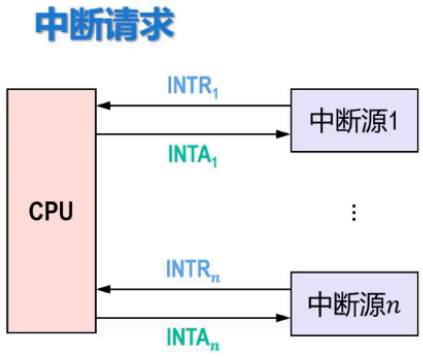
一、独立请求方式

独立请求方式

链式请求方式

分组链式请求方式

中断控制器方式



优 点

- ✓ CPU可以**灵活处理中断优先级**。
- ✓ **中断识别简单**，通过简单的优先级编码电路实现。
- ✓ **方便实现向量中断**，可快速找到中断服务程序。

缺 点

- □ **占用大量CPU引脚**作为INTR信号线和INTA信号线。
- □ **系统难以扩展**。

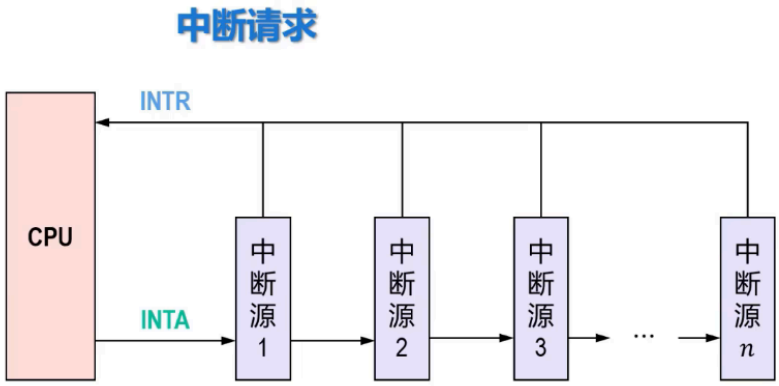
二、链式请求

独立请求方式

链式请求方式

分组链式请求方式

中断控制器方式



- 各中断源通过**一条公共的中断请求INTR线**向CPU传送中断请求信号。
- **中断源的识别**过程比较**复杂**。
  - **硬件查询方法**：通过**中断应答INTA信号**的**链式串行传递**（与总线仲裁的链式查询方式相同）实现。在链式串行传递的路径中，离CPU越近的中断源优先级越高。
  - **软件查询方法**：通过**公共的中断服务程序轮询**各中断源的I/O接口中的寄存器，判断相应设备是否发出了中断请求，若发出了请求则跳转至相应的中断服务程序，若没有发出请求则继续查询下一个中断源。中断优先级取决于设计在公共中断服务程序中的轮询顺序，先查询的设备优先级高。

类似：[[7.5 总线仲裁]]

优缺点

优 点

✓ **系统扩展性好**。添加中断源只需接入公共的中断请求INTR线，无需额外布线，硬件实现简单。

缺 点

- 中断源识别过程复杂。
- 硬件查询方法：中断优先级由设备在链式串行传递中的物理位置固定，无法通过软件灵活调整。
- 软件查询方法：需逐个轮询设备状态寄存器（需要CPU发起相应的总线事务）。

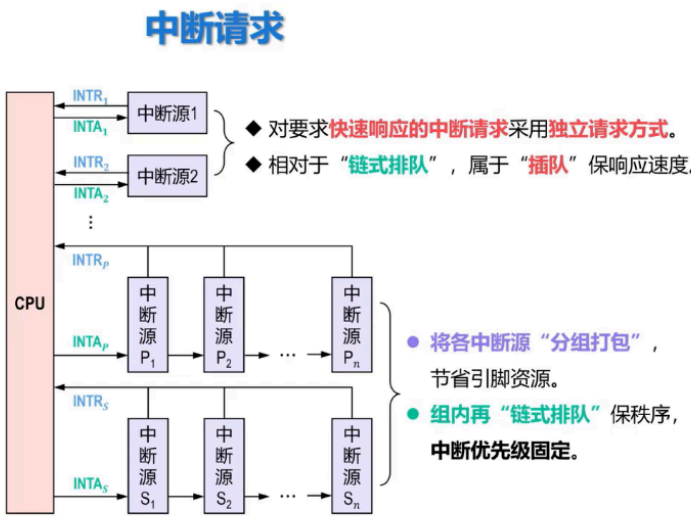
三、分组链式请求方式

独立请求方式

链式请求方式

分组链式请求方式

中断控制器方式



分组链式请求方式通过“二维分层”，用结构设计**平衡了引脚资源消耗**与中断**响应速度**。

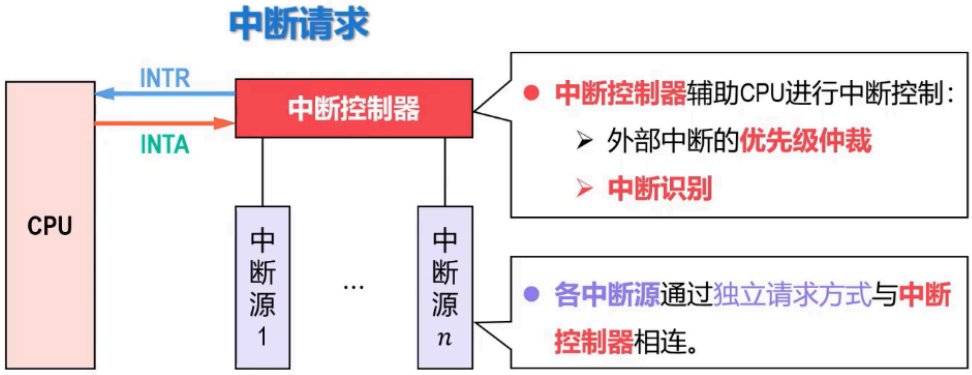
四、中断控制器方式

独立请求方式

链式请求方式

分组链式请求方式

中断控制器方式



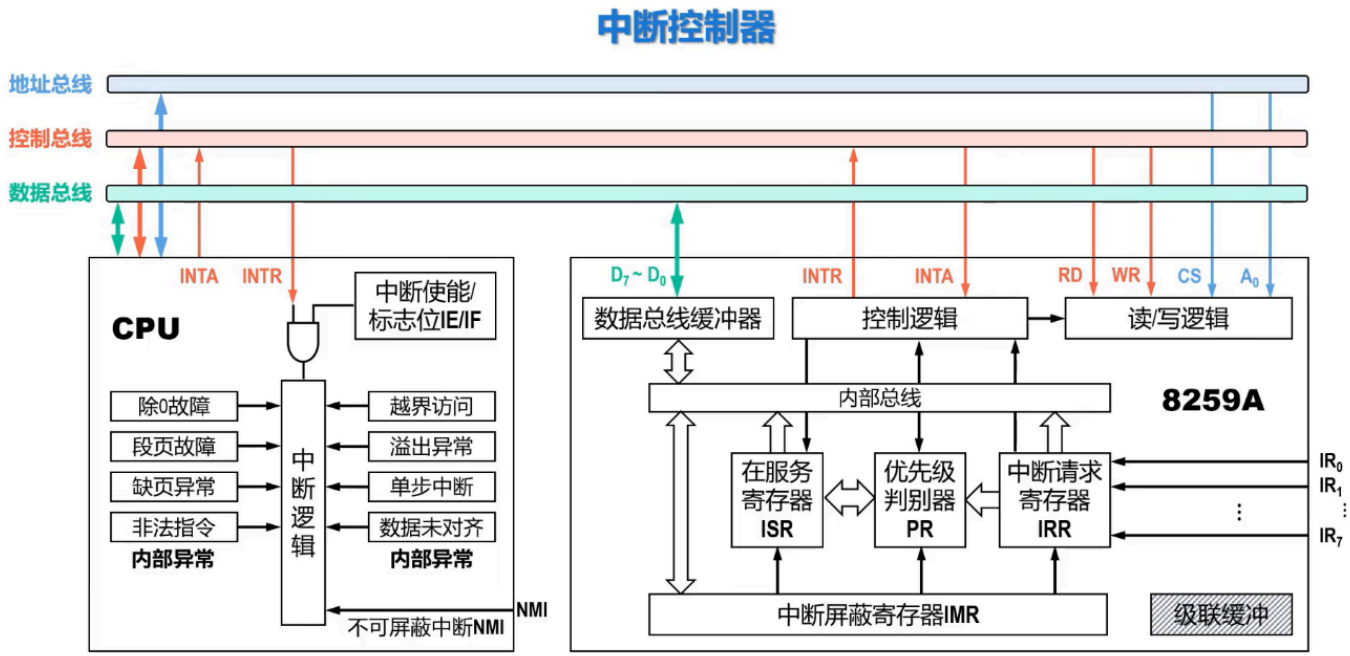
① 有中断源发出中断请求。

② 中断控制器进行中断优先级仲裁和中断识别后，通过中断请求INTR信号线向CPU发送中断请求。

③ CPU通过中断响应INTA信号线给中断控制器发送中断响应。

④ 中断控制器将中断源编号通过**数据总线**（图中未画出）发送给CPU。

4.1 中断控制器硬件逻辑图举例



PU侧中断逻辑：内中断与外中断的分流（图的左半部分）

进入CPU的中断请求分为两大类，这是408反复考的分类题：

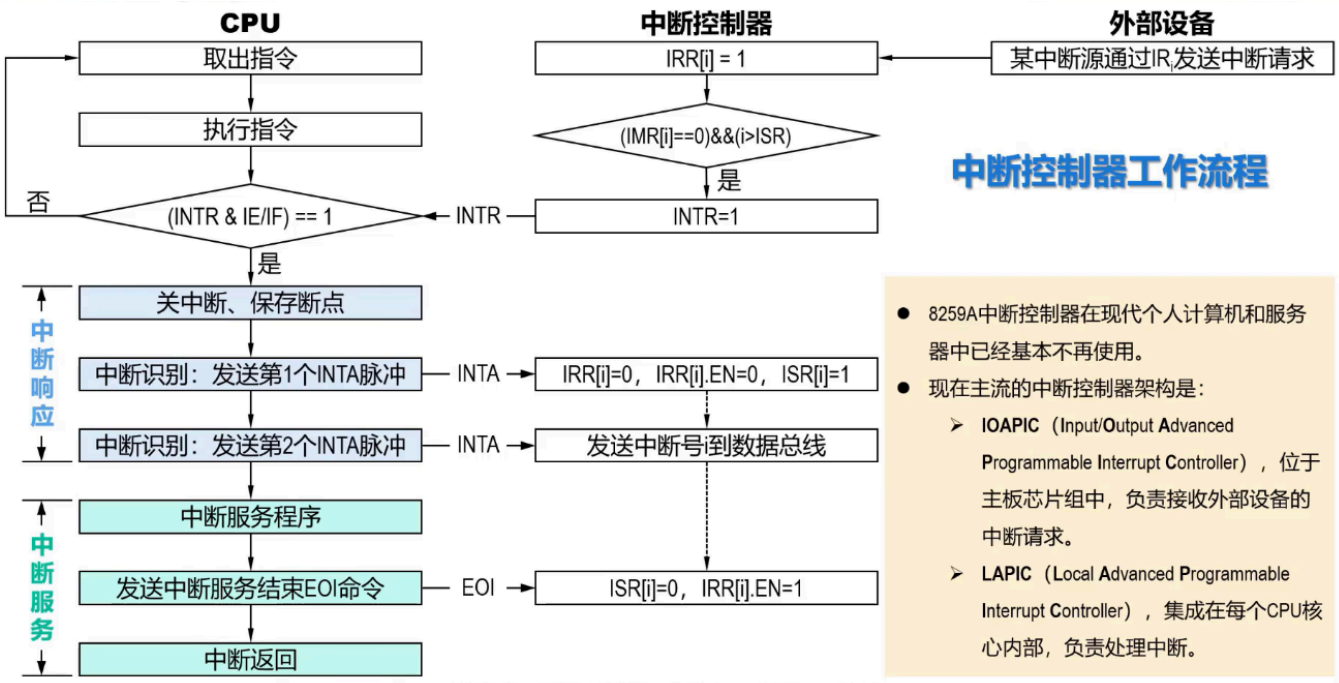
分类维度	内中断（异常）	外中断（中断）
触发来源	CPU执行指令**内部**产生	CPU**外部**设备发出
是否与指令相关	同步，与当前指令绑定	异步，与指令无关
是否可屏蔽	一律**不可屏蔽**	INTR可屏蔽（受IF位控制），NMI不可屏蔽
典型例子	除0故障、非法指令、溢出、缺页、单步、越界、数据未对齐	键盘/打印机等I/O中断、时钟中断、掉电（NMI）
检测时机	指令执行过程中检测	每条指令**执行完毕**的边界检测
图里CPU左侧那几根线（除0、段页故障、缺页、非法指令、越界、溢出、单步、数据未对齐）全部汇入"中断逻辑"，对应**内中断（异常）**，其中除0/溢出/非法指令/越界/缺页属于"故障"或"终止"，单步属于"自陷"。右侧经**NMI**直接进入的是不可屏蔽外中断（如掉电、硬件故障），**不经过8259A**；经**INTR**进入的才是8259A送来的可屏蔽外中断，且要经过"中断使能/标志位IF"这个与门的过滤——**IF=0时，所有INTR信号都被挡掉，但NMI和内中断不受影响**		
#### 8259A内部核心部件：四个寄存器/部件的分工（图的右半部分）		
8259A是CPU与外设之间的"中断管理者"，单片管理8级中断，通过级联最多扩展到64级。理解图的重点是这四个核心部件**各自负责什么、以及信号在它们之间的流动顺序**：		

部件	全称	作用	关键特征
IRR	中断请求寄存器	锁存IR0~IR7上到达的中断请求信号	对应位置1；被响应后该位才清0
IMR	中断屏蔽寄存器	过滤IRR中的请求，被屏蔽位对应的中断不能进PR	由CPU通过OCW1设置，每位独立屏蔽
PR	优先级判别器	从未被屏蔽的IRR请求中选优先级最高者，并与ISR中"正在服务"的中断比较	IR0优先级最高，IR7最低；支持嵌套判优
ISR	在服务寄存器	记录当前CPU正在服务的中断	响应后由INTA置位，EOI后清0
信号流顺序（必须背下来）：设备请求 → IRR锁存 → IMR过滤 → PR判优 → 通过INT向CPU的INTR发请求 → CPU回INTA → ISR对应位置1、IRR对应位清0 → 第二个INTA期间8259A把**中断类型号**放到D7~D0数据总线上 → CPU据此查中断向量表跳转ISR			
#### 完整的可屏蔽中断响应流程			
一次外中断从请求到返回，按时间顺序分四段：			
① 中断请求与判优（8259A内部）			
设备在IRn上发请求 → IRR置位 → IMR判断是否屏蔽 → PR比较IRR中未屏蔽请求与ISR中正在服务的中断的优先级 → 若更高，向CPU发INT信号			
② 中断响应（硬件自动完成，即"中断隐指令"）			
CPU在**一条指令执行完毕**且**IF=1**时响应，硬件自动完成三件事：			
- **关中断**（清IF，防止保护现场被打断）			
- **保存断点**（压PC入栈）			
- **取向量、跳转ISR入口**（从数据总线D7~D0取回类型号 → 查中断向量表 → 入口地址送PC）			
③ 中断服务（软件完成，ISR内部）			
保护现场（压通用寄存器和PSW） → 必要时开中断允许嵌套 → 执行具体处理 → 发EOI命令给8259A（清ISR对应位） → 关中断 → 恢复现场 → 执行IRET中断返回			
④ 中断返回			
IRET硬件自动弹出断点和PSW，回到被打断的程序			
#### 408高频易错点提醒			

部件	全称	作用	关键特征
以下几个点是历年真题反复"挖坑"的地方，需要重点辨析：			
- **第一，"保存断点"和"保护现场"的执行主体不同。** 保存断点（PC）是**硬件在中断隐指令阶段自动完成**的；保护现场（通用寄存器、PSW）是**软件在ISR里手动压栈完成**的。2010-21真题就专门考过ISR内部的执行顺序，正确答案是 I→VI→VII→II→V（保护现场→中断处理→恢复现场→开中断→中断返回），关中断和保存断点不属于ISR内部操作			
- **第二，"中断向量"和"向量地址"是两个概念。** 中断向量是**中断服务程序的入口地址**本身；向量地址是**中断向量表中存放该入口地址的内存单元地址。**CPU拿到的是向量地址，查表后才得到入口地址			
- **第三，IF位只对INTR有效。** IF=0屏蔽的是所有经8259A/INTR来的可屏蔽中断，但NMI（不可屏蔽中断）和所有内中断（异常）**不受IF位控制**，仍然会被响应			
- **第四，INTA是两次脉冲。** CPU响应后第一个INTA通知8259A"请求被接受"（此时ISR置位、IRR清零），第二个INTA期间8259A才把**中断类型号**放到数据总线上。这两步不能合并，是判断流程正误的关键细节			
- **第五，中断响应的三个必要条件缺一不可。** 有未被屏蔽的中断请求、CPU开中断（IF=1）、**一条指令执行完毕。**CPU绝不会在一条指令执行中途响应可屏蔽外中断，这是指令级原子性			
- **第六，优先级判别是"双向比较"。** PR不仅要在多个新请求中排队，还要把新请求与ISR中"正在服务"的中断比较，只有新请求优先级更高才会向CPU再次发出INT——这就是**中断嵌套**的硬件基础。			
- **第七，EOI的时机影响嵌套。** 如果ISR处理完没发EOI命令，ISR对应位一直保持1，PR会认为该中断"还在服务"，后续同级或更低级的中断无法被响应。这是"中断结束方式"考点的核心			

把握住"**IRR→IMR→PR→INT→INTA→ISR→类型号→向量表→ISR执行→EOI→IRET**"这条主线，再对照图里的箭头走一遍，整张图就串起来了

4.2 中断控制器工作流程图举例



按照"设备发请求 → 8259A把关 → CPU响应 → 执行服务 → 收尾返回"这五幕剧来看，四根横跨CPU与8259A之间竖线的信号（INTR、INTA、INTA、EOI）就是幕与幕之间的"交接棒"

第一幕：请求诞生（图右上角 + 中间列顶部）

某个外部设备（比如键盘）在自己的中断请求线 **IRi** 上拉高电平。8259A做的第一件事只是**登记**：把中断请求寄存器的对应位置1，即 **IRR[i] = 1**

注意这一步的微妙之处：此时请求只是被8259A"记录在案"，**CPU完全不知道**。设备不会直接敲CPU的门，一切都要先过8259A这一关

第二幕：8259A的层层把关（中间列的菱形判断）

8259A持续对IRR里的请求做两个判断，对应图中菱形 `(IMR[i]==0) && (i>ISR)`：

1. **过IMR这一关**：`IMR[i]==0` 表示该中断源没有被屏蔽。如果CPU之前通过OCW1把这位屏蔽了，请求就死在这里，永远到不了CPU

2. **过PR这一关**：`i>ISR` 是"请求i的优先级高于ISR中正在服务的中断"的简化写法。这里出现了**ISR（在服务寄存器）**——它记录着CPU当前正在处理谁。新请求只有"打赢"正在服务的那一个，才值得去打扰CPU；如果CPU正忙于一个更高优先级的中断，新请求只能在IRR里排队等着

两关都过了，8259A才在 **INTR** 线上置1，向CPU发出正式申请。**这就是第一根跨线信号：INTR（8259A → CPU）**

第三幕：CPU的日常循环与响应时机（左列上方的环）

与此同时，CPU在干自己的本职工作，图左侧那个大回环就是CPU的永恒循环：

> 取出指令 → 执行指令 → 检查 `(INTR & IE/IF) == 1`？ → 否，回到取出指令
> 这个菱形里的条件非常有讲究，是**两个条件的逻辑与**：
> - **INTR**：外面真的有人申请吗？
> - **IE/IF**：CPU自己开中断了吗？IF=0时，即使INTR=1也视而不见。
> 关键在于这个检查的**时机**——它发生在"一条指令执行完"的边界上，CPU绝不会在指令执行到一半时被可屏蔽中断打断。所以从设备拉高IRi到CPU真正响应，中间可能隔了很多条指令的时间，中断请求就在IRR里耐心等着
> 条件满足，CPU沿"是"的分支进入**中断响应**阶段——注意图中这一段（蓝色框）全部是**硬件自动完成**的，不需要任何软件参与，这就是"中断隐指令"

第四幕：中断响应——两次INTA的握手（本图最精华的部分）

第一步：关中断、保存断点。 CPU先清IF位（防止响应过程自身再被新中断打断，保证保存断点的原子性），然后把断点（PC）压栈。这一步完全在CPU内部完成

第二步：发第1个INTA脉冲。 这相当于CPU对8259A说："你的申请我批准了。"8259A收到后做三件内部状态更新：

- `IRR[i]=0`：撤销挂起的请求（已受理，不用再等了）
- `ISR[i]=1`：登记"正在服务中"（这就是第二幕PR判优的依据来源，形成闭环）
- `IRR[i].EN=0`：封锁该中断源，服务期间不允许它再次申请

第三步：发第2个INTA脉冲。 这相当于CPU问："那么，你是谁？"8259A把**中断类型号**放上数据总线，CPU取回后据此查中断向量表，得到中断服务程序的入口地址送入PC。到此为止，CPU"知道该跳去哪了"

INTA必须发两次脉冲，第一次是"确认+状态切换"，第二次才是"交付类型号"，这是408判断流程正误的经典考点

第五幕：中断服务与收尾（左列下方的绿色框）

从这一刻起，**控制权交给软件。**CPU开始执行**中断服务程序**：保护现场（压通用寄存器），必要时开中断（允许更高优先级的请求实现嵌套——硬件基础正是第二幕里PR对ISR的比较），然后干正事（比如读走键盘缓冲区的数据）

收尾阶段有两件事，顺序不能乱：

① 发送EOI命令。 CPU向8259A的端口写一个EOI（End Of Interrupt，中断结束）命令，8259A收到后：

- `ISR[i]=0`：解除"正在服务"登记
- `IRR[i].EN=1`：重新放开该中断源

如果忘了发EOI，ISR[i]永远是1，PR会一直认为"这个中断还在服务"，同级和更低级的中断从此石沉大海——这就是为什么EOI单独作为一个框出现在流程里，它是软件和8259A之间的明确交接

② 中断返回。 IRET指令自动弹回断点和PSW，控制权回到主程序，图中的箭头绕回"取出指令"，CPU回到那个永恒循环。一次中断的完整生命周期结束

一张表串起四根“跨线信号”

信号	方向	时机	一句话含义
INTR	8259A → CPU	过了IMR+PR两关后	"有人申请，且优先级够格"
INTA 第1脉冲	CPU → 8259A	关中断保存断点之后	"批准了"→8259A清IRR、置ISR
INTA 第2脉冲	CPU ← 8259A (经数据总线)	紧接第1脉冲	"你是谁"→送中断类型号
EOI	CPU → 8259A	服务程序末尾	"干完了"→清ISR、解封中断源
#### 两个延伸理解			
关于中断嵌套：这张图里 (i>ISR) 的判断出现在INTR发出**之前**，也就是说嵌套的准入门槛是8259A把的——正在服务小任务时来了大任务，8259A会再次拉高INTR；而CPU侧能否真正嵌套，还要看ISR里是否及时开了中断（本图为简化未画出开中断这一步，实际服务程序在保护现场后会STI）。软硬件两侧条件同时满足，嵌套才发生			
关于图右下角的备注：8259A在现代计算机中已基本退役，取而代之的是**IOAPIC**（位于芯片组，收外部设备的中断）+ **LAPIC**（每个CPU核一个，处理中断）的两级架构。但408考纲仍以8259A为教学模型，因为它把“判优、屏蔽、嵌套、向量”这些中断系统的本质机制展现得最完整——学懂了8259A的工作流，IOAPIC/LAPIC只是把这套逻辑分布式化了而已			
一句话总结整张图：**设备登记（IRR）→ 8259A把关（IMR+PR）→ 举手（INTR）→ CPU择机批准并两次握手（关中断/断点 + 2×INTA）→ 软件干活（ISR）→ 打勾销案（EOI）→ 回家（IRET）**。顺着这条主线在图上把箭头走一遍，整个中断控制器的运行逻辑就完全串起来了			